

NAMOSIM: a Robot Motion Planner for Navigation Among Movable Obstacles

David Brown^{1,4}, Jacques Saraydaryan^{1,3,4}, Benoit Renault^{2,4}, and Olivier Simonin^{1,2,4}

1 Inria, CHROMA Team 2 INSA Lyon 3 CPE Lyon 4 CITI Laboratory

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- Review [↗](#)
- Repository [↗](#)
- Archive [↗](#)

Editor: [Open Journals](#) [↗](#)

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

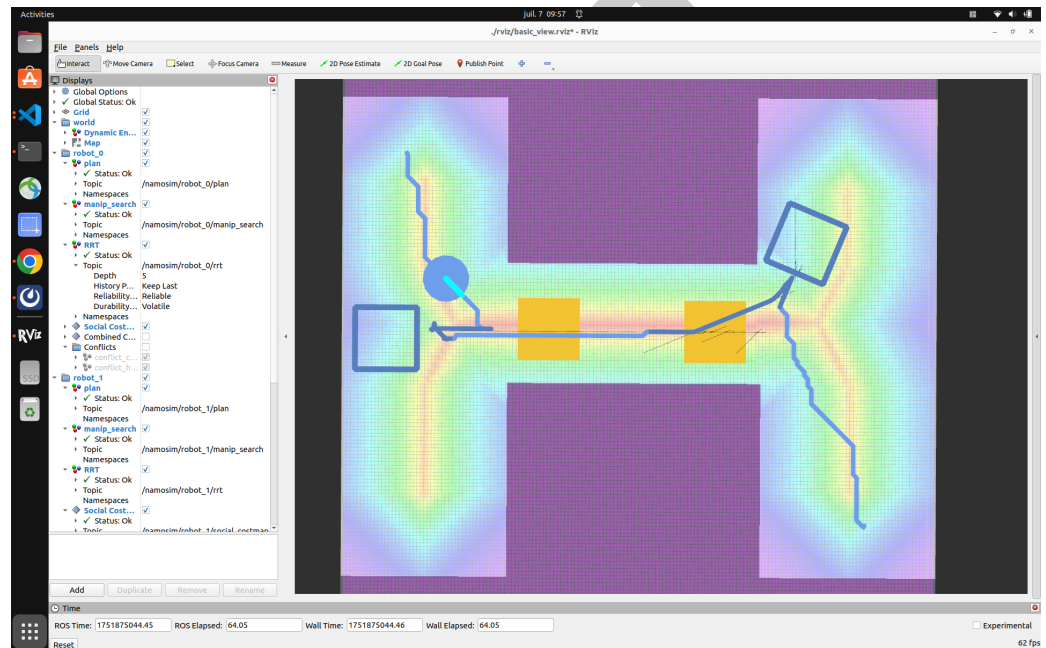


Figure 1: NAMOSIM

Summary

NAMOSIM is a mobile robot motion planner designed for the problem of Navigation Among Movable Obstacles (NAMO). The planner simulates robots navigating in 2D polygonal environments where certain obstacles can be grasped and relocated to enable robots to reach their goals. NAMOSIM thus extends the classic navigation problem with a layer of interactivity, posing interesting research questions while remaining well-defined and amenable to both classical and learning-based approaches. NAMOSIM is intended for researchers and developers working on robot navigation in dynamic environments, particularly where physical interaction is necessary.

NAMOSIM supports the development of novel NAMO algorithms using a modular agent-based architecture. It includes a baseline NAMO algorithm (Stilman & Kuffner, 2005) implemented in our Stilman2005 agent, which also incorporates a communication-free coordination strategy for multi-robot scenarios. A variety of other agent types are implemented, and new agents utilizing alternative approaches can be created and integrated into the planner by implementing the **Agent** base class. NAMOSIM thus supports reproducible research in NAMO algorithms, including those based on machine learning and AI.

NAMOSIM is packaged as a ROS2 package for easy integration into robotics projects and can also be used as a standalone Python module. It utilizes ROS2 messages for visualization of environments, plans, and execution via RViz and includes several prebuilt scenarios for testing and benchmarking. Scenarios are stored as SVG files, allowing the convenient creation of custom scenarios using a free SVG editor such as Inkscape.

Statement of Need

Many interesting applications in autonomous mobile robotics involve some kind of physical interaction with the environment as well as social coordination with other agents. However, global navigation planners typically assume static, non-interactive environments, leaving higher-level behaviors to other parts of the robot software stack and thus complicating their implementation. This limits the applicability of the standard navigation stack in complex real-world scenarios. Ideally, motion planners should be able to reason about physical and social interactions. NAMOSIM addresses this gap by offering a simulation environment explicitly designed to study NAMO problems, which involve not only path planning but also reasoning about which obstacles to move, where to move them, and how to combine standard navigation with obstacle manipulation. Additionally, NAMOSIM supports multi-robot environments, facilitating reproducible research in social navigation.

Major Features

NAMOSIM provides a robust set of features to support research and development in Navigation Among Movable Obstacles (NAMO):

- **Modular Agent-Based Architecture:** The simulator is built around a flexible Agent interface, allowing users to implement and test custom NAMO planning algorithms. A baseline NAMO algorithm is included in the Stilman2005 agent for immediate use and benchmarking.
- **Support for Multiple Robot Models:** NAMOSIM supports both holonomic and differential-drive robot models, enabling realistic simulation of various robotic platforms.
- **ROS2 Integration:** NAMOSIM forms a ROS2 package, enabling seamless integration into simulated and physical robotics projects and visualization via RViz.
- **2D Environment Simulation:** The simulator provides a customizable 2D environment where users can define static and movable obstacles, supporting complex scenarios for testing multi-robot coordination strategies and NAMO algorithms.
- **Prebuilt Scenarios and Tests:** NAMOSIM includes several custom scenario files for benchmarking and testing specific situations.
- **Multi-Robot Coordination:** The simulator supports multi-robot scenarios, and our baseline Stilman2005 agent implements the communication-free coordination strategy presented in our IROS-2024 publication (Renault et al., 2024).

These features make NAMOSIM a versatile tool for prototyping, evaluating, and deploying NAMO algorithms in diverse robotic applications.

Customizable Scenarios

NAMOSIM environments, or **scenarios**, are stored in SVG format and can be edited using any SVG editor, such as Inkscape. The scenario SVG file contains the following key elements:

- The geometry of the static map
- The polygons and orientations of all robots and movable obstacles
- Configuration settings that define the behavior of the environment and robots

66 The static map can also be included as an image layer within the SVG to conveniently
67 incorporate ROS grid-map images generated by standard mapping tools.

68 Architecture

69 At a high level, NAMOSIM executes a SENSE-THINK-ACT loop that performs the following
70 functions at each iteration:

- 71 1. **SENSE**: Each agent senses the environment and updates its internal representation.
- 72 2. **THINK**: Each agent computes a new plan or updates its current plan.
- 73 3. **ACT**: Each agent selects a single discrete action to execute.

74 The loop is expected to execute at a regular frequency, with the assumption that all agent
75 functions run sequentially in a synchronized manner.

76 Stilman's Algorithm

77 NAMOSIM includes a baseline implementation of Stilman's 2005 NAMO algorithm ([Stilman
78 & Kuffner, 2005](#)). The key idea of this algorithm is to move obstacles to merge disjoint
79 components of the robot's free configuration space. The map is divided into a set of disjoint
80 **connected components**, where each cell in a given component is reachable from all other cells
81 in the same component. It can be proven that components are separated by movable obstacles
82 or are otherwise unreachable. The algorithm functions by moving obstacles to join components
83 until the robot's current component includes the goal cell.

84 The algorithm works by recursively performing the following two stages:

- 85 1. **SELECT_OBSTACLE_AND_COMPONENT**: The first stage performs a simplified
86 A* grid search, allowing the agent to pass through movable obstacles. It returns the ID
87 of the first movable obstacle encountered on the optimal path to the goal and the ID of
88 the component encountered after passing through the obstacle.
- 89 2. **OBSTACLE_MANIPULATION_SEARCH**: The second stage finds a **transit path**
90 from the robot's current position to a grasp pose near the obstacle. Then, it finds a
91 **transfer path** by performing an obstacle manipulation search to join the robot's current
92 component to the component selected in stage 1. If this stage fails, the obstacle and
93 component pair are added to an avoid-list, and the algorithm returns to stage 1.

94 Each iteration of the algorithm continues with a copy of the environment where the robot and
95 obstacle start from the poses resulting from the previous obstacle manipulation search. This
96 algorithm is explained in greater detail in Renault's 2023 PhD thesis ([Renault, 2023](#)).

97 Collision Detection

98 Custom agents can implement their own collision detection routines; however, the baseline
99 Stilman2005 agent detects collisions using a simple binary-occupancy grid during transit paths
100 (when not carrying an obstacle), assuming a circular robot footprint. When transporting a
101 movable obstacle, the robot footprint is non-circular, and collision detection is based on the
102 **convex swept volume** resulting from the area swept by the combined robot-obstacle footprint
103 due to the action motion. Although computationally expensive, this ensures all possible
104 collisions are detected, regardless of the shape of the robot or obstacle.

105 Social Costmap

106 A novel contribution in the baseline implementation is the option to use a social costmap
107 during the obstacle manipulation search to guide obstacle placement decisions. This allows
108 robots to place obstacles in areas less likely to block the free passage of other agents, including
109 humans, reducing the likelihood that obstacles will need to be moved again. The key heuristic

of the social costmap is to **avoid narrow corridors and central areas**, assigning higher costs to narrow corridors and the centers of open spaces. This helps robots avoid placing obstacles in front of doorways or in the center of rooms. The social costmap is explained in greater detail in Renault (2023).

Conflict Avoidance and Deadlock Resolution

The baseline Stilman2005 agent can avoid conflicts and resolve deadlocks with other agents. Conflict avoidance works by looking ahead along the agent's current plan for a fixed number of steps, called the **conflict horizon**. Within this horizon, the agent simulates each planned action and checks for potential conflicts. For example, the agent may have planned to move an obstacle that is no longer at the expected location, or another robot may be crossing the planned path within the conflict horizon, raising the potential for a collision.

Our Stilman2005 agent avoids conflicts by either pausing or replanning around them. A **deadlock** is detected when the same conflict configuration is repeatedly encountered, even after replanning. To resolve deadlocks, the agent follows an evasion strategy as described in our IROS 2024 publication (Renault et al., 2024).

Acknowledgements

This research was supported by the Inria NAMOEX initiative. We express our gratitude to Benoit Renault, whose PhD thesis forms the foundation of this work.

References

- Renault, B. (2023). *NAvigation en milieu MODifiable (NAMO) étendue à des contraintes sociales et multi-robots* (Theses No. 2023ISAL0105, INSA de Lyon). <https://hal.science/tel-04418723>
- Renault, B., Saraydaryan, J., Brown, D., & Simonin, O. (2024). Multi-robot navigation among movable obstacles: Implicit coordination to deal with conflicts and deadlocks. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 1–7. <https://hal.science/hal-04705395>
- Renault, B., Saraydaryan, J., & Simonin, O. (2020). Modeling a social placement cost to extend navigation among movable obstacles (NAMO) algorithms. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 11345–11351. <https://doi.org/10.1109/IROS45743.2020.9340892>
- Stilman, M., & Kuffner, J. J. (2005). Navigation among movable obstacles: Real-time reasoning in complex environments. *International Journal of Humanoid Robotics*, 2(4), 479–503. <https://doi.org/10.1142/S0219843605000545>